

A PROPOSAL

Know what's in what we ship

pixi-sbom writes a software bill of materials for every environment we build. One job in CI, and we can answer the question.

What I'm asking for

Let me add one job to one repo's CI. Two weeks later I'll show you what came out of it, and we decide from there.

Effort

One workflow file. Nothing to host and nothing to operate.

Cost

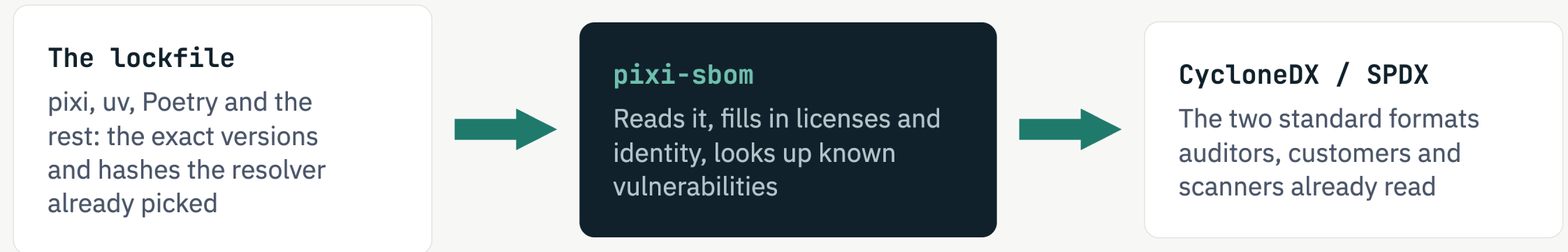
No license fee, no service, no account. It's Apache-2.0.

Risk

It reads the lockfile and writes a file. It can't break a build unless we ask it to.

What a software bill of materials is

A machine-readable list of everything in a build: names, versions, licenses, who supplied each piece, and what depends on what. The ingredients label for software.



Why we're being asked for this now

Rule	Who it covers	What it asks for	Since
PCI DSS 4.0, Req. 6.3.2	Banks and anyone else handling card data	An inventory of our own software and the third-party components in it	31 Mar 2025
EU DORA, RTS 2024/1774 Art. 10	EU banks, insurers and investment firms	Track third-party libraries, open source included, in services behind critical functions	17 Jan 2025
EU Cyber Resilience Act	Anyone selling software in the EU	A machine-readable SBOM for each product	11 Dec 2027
US Executive Order 14028	Software sold to federal agencies	An SBOM when an agency asks, with NTIA's minimum elements	2021
Customers	Every deal with a security review	Questionnaires we answer by hand today	Now

The two financial rules already apply. Neither says "SBOM"; an SBOM is how we show the inventory they require.

Why the tools already out there don't fit

Most SBOM tooling was built for pip, npm and Maven. A pixi environment is conda packages **and** PyPI wheels, resolved together into one lockfile.

What a generic tool sees

- The wheels it recognizes
- Conda packages as unknown, or missed entirely
- No channel, build string or subdir
- An inventory that looks complete and isn't

What pixi-sbom sees

- Every locked package, both kinds
- Channel, build, subdir and hashes
- PyPI identity matched onto conda packages
- The dependency graph the resolver picked

What it is

One binary. It reads the lockfile `pixi`, `uv`, `Poetry`, `PDM`, `conda-lock` or `pip` wrote, and writes CycloneDX or SPDX.

12 MB

one executable, no
runtime to install

5

platforms: Linux, macOS,
Windows, Intel and ARM

0

servers, accounts or
telemetry

Offline

works with no network
when it has to

What it does



Both standards

CycloneDX 1.6 and 1.7, SPDX 2.3 and 3.0.1. Whoever asks gets the format they already use.



Known vulnerabilities

Looks them up in OSV, flags CISA's known-exploited list, and ranks the rest by how likely exploitation is (EPSS).



License rules

Allow and deny lists, with exemptions that have to carry a reason.



Provenance

Every fact records where it came from, including the sources that came back empty.



Supplier SBOMs

Grades a vendor's document before we trust it, applies their VEX, and merges it with ours into one.



Any Python project

pixi, uv, Poetry, PDM, conda-lock or pinned pip. From PyPI, a GitHub Action or a pre-commit hook.

What it isn't

Not a vulnerability scanner

It tells you what you're shipping and hands that to whatever scans it. It doesn't replace the security tooling we already pay for.

Not a policy engine for the whole company

It enforces license and vulnerability rules for one build. Governance across every team is a different problem with different owners.

Not a promise that nothing is missing

When an upstream publishes no license or no advisory, the document says so instead of filling in a guess. You can tell an unanswered question from an empty answer.

What a team gets out of it

How it goes today	With this in CI	Signal
Someone asks if we're exposed to a CVE	Search the document for that release	Minutes, not a day
A copyleft license rides in on a transitive dependency	The build stops before it ships	Exit 3
A dependency is known exploited, or likely to be	The build stops and names the finding	Exit 4
Code imports a package nobody declared	Reported against the manifest	Exit 8
"What changed since the last release?"	A diff of the two documents	Exit 6, if we gate it
A supplier sends an SBOM too thin to check	Graded before anyone trusts it	Exit 10

Every gate is off by default. Leave them off and it writes the document and exits clean.

What it costs us

1 file

A step in a workflow. Nothing to host or keep running.

< 1 s

Added to a build, on a 72-package environment, offline.

\$0

Apache-2.0. No seats, no tiers, nothing to renew.

The real cost is somebody reading the output now and then, and that's work we're already being asked to do.

Timings from the project's published benchmarks, measured on all five platforms at every release.

Where the project is

- 🔍 Version 1.8.1, with a changelog and signed release builds for five platforms
- 📦 Installs the way our other tools do: [pixi global](#), [pip](#) or [uvx](#)
- 📖 A docs site with a field manual and a training lab, and a diagnostic mode for locked-down networks
- 📊 Speed and memory measured on all five platforms at every release, numbers published
- 🔒 Since 1.0 the flags, exit codes and output fields are under a written stability promise
- 👥 Apache-2.0, so we can read it, fork it, or fix it ourselves

WHAT HAPPENS NEXT

One repo. One job. Two weeks.

I'll come back with the documents it produced, what the gates would have caught if they'd been switched on, and whether I think it's worth doing anywhere else.